

CS57300
PURDUE UNIVERSITY

DATA MINING

PRINCIPLE COMPONENT ANALYSIS (PCA)

- ▶ Input: the $n \times p$ data matrix $\mathbf{D}$
- ▶ Preprocess $\mathbf{D}$ so that the mean of each dimension is 0, call this matrix $\mathbf{X}$
- ▶ Goal: Find a $p \times p$ orthogonal transformation matrix $\mathbf{A}$ to conduct basis change, i.e., $\mathbf{Y} = \mathbf{XA}$, such that under the new basis, the covariances between the new dimensions are low
 - ▶ The $p \times p$ covariance matrix $\mathbf{Y}^T \mathbf{Y}$ is a diagonal matrix.

PRINCIPLE COMPONENT ANALYSIS (PCA)

$$\begin{aligned} Y^T Y &= (XA)^T (XA) \\ &= A^T X^T X A \end{aligned}$$

- ▶ Notice $\Sigma = X^T X$ is the covariance matrix under the current basis, it is a symmetric square matrix!
- ▶ So, we can conduct eigendecomposition for Σ

$$Y^T Y = A^T (Q \Lambda Q^T) A = (A^T Q) \Lambda (A^T Q)^T$$

PRINCIPLE COMPONENT ANALYSIS (PCA)

$$Y^T Y = A^T (Q \Lambda Q^T) A = (A^T Q) \Lambda (A^T Q)^T$$

- ▶ Let $A^T Q = I$, we get $A = (Q^{-1})^T = (Q^T)^T = Q$
 - ▶ By doing so, $Y^T Y = I \Lambda I = \Lambda$
- ▶ In other words, the transformation matrix A is Q , where each column is the eigenvector of the covariance matrix $\Sigma = X^T X$!
- ▶ The column vectors of A (or Q) are thus called the **principle component vectors**!

NOT DONE YET...

- ▶ So far we have only changed basis, i.e., we project the data to another p dimensions
 - ▶ The covariance on these p new dimensions is 0!
 - ▶ But we don't know which dimensions we can drop, i.e., which dimensions have smaller variance yet...
- ▶ Recall that $Y^T Y = \Lambda$, this is the covariance matrix after projecting data to the p new dimensions!
 - ▶ λ_i is the variance of new dimension i (i.e., the i -th column of A), $\sum_{j=1}^p \sigma_j^2 = \sum_{j=1}^p \lambda_j$

APPLYING PCA

- ▶ Order principal components according to the corresponding eigenvalues. New data vectors are formed by projecting the data onto the first few principal components (i.e., top m eigenvectors)

$$\mathbf{x} = [x_1, x_2, \dots, x_p] \text{ (original instance)}$$

$$\mathbf{A} = [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_p] \text{ (principal components)}$$

$$x'_1 = \mathbf{a}_1 \mathbf{x} = \sum_{j=1}^p a_{1j} x_j$$

...

$$x'_m = \mathbf{a}_m \mathbf{x} = \sum_{j=1}^p a_{mj} x_j \quad \text{for } m < p$$

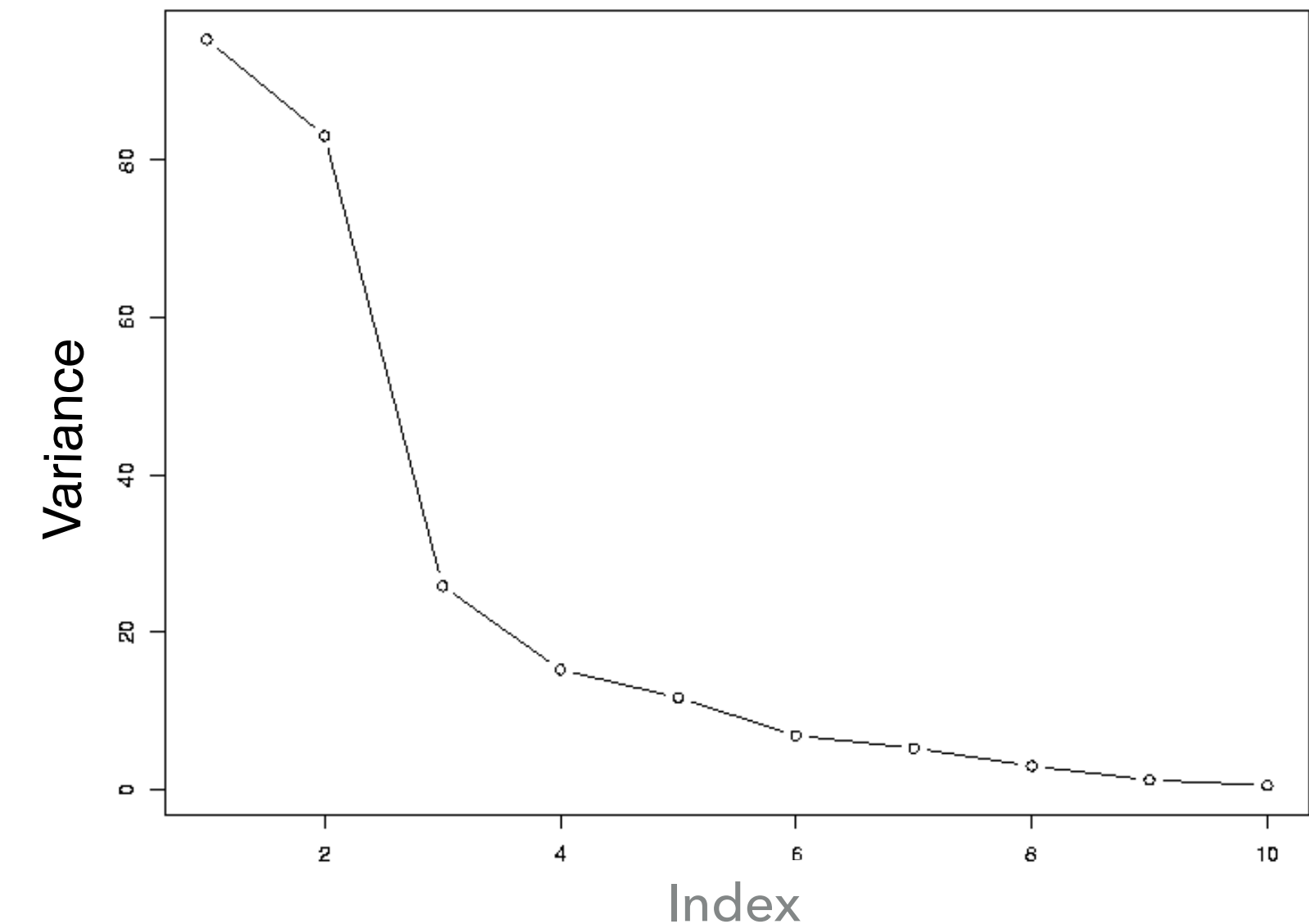
If $\mathbf{m}=\mathbf{p}$ then data is transformed

If $\mathbf{m}<\mathbf{p}$ then transformation is lossy
and dimensionality is reduced

$$\mathbf{x}' = [x'_1, x'_2, \dots, x'_m] \text{ (transformed instance)}$$

APPLYING PCA (CONT')

- ▶ Goal: Find a new (smaller) set of dimensions that captures most of the variability of the data
- ▶ Use **scree plot** to choose number of dimensions
 - ▶ Choose $m < p$ so projected data captures much of the variance of original data



EXAMPLE: EIGENFACES

PCA applied to images of human faces.

Reduce dimensionality to set of basis images.

All other images are linear combo of these “eigenpictures”.

Used for facial recognition.



First 40 PCA dimensions

PREDICTIVE MODELING

DATA MINING COMPONENTS

- ▶ Task specification: **Prediction**
- ▶ Knowledge representation
- ▶ Learning technique
- ▶ Prediction and/or interpretation

PREDICTIVE MODELING

- ▶ Data representation:
 - ▶ Paired attribute vectors and class labels $\langle y(i), \mathbf{x}(i) \rangle$ or $n \times p$ tabular data with class label (y) and $p-1$ attributes ($\mathbf{x}$)
- ▶ Task: Estimate a predictive function $f(\mathbf{x}; \theta) = y$
 - ▶ Assume that there is a function $y = f(\mathbf{x})$ that **maps** data instances ($\mathbf{x}$) to class labels (y)
 - ▶ Construct a model that approximates the mapping
 - ▶ Classification: if y is categorical
 - ▶ Regression: if y is real-valued



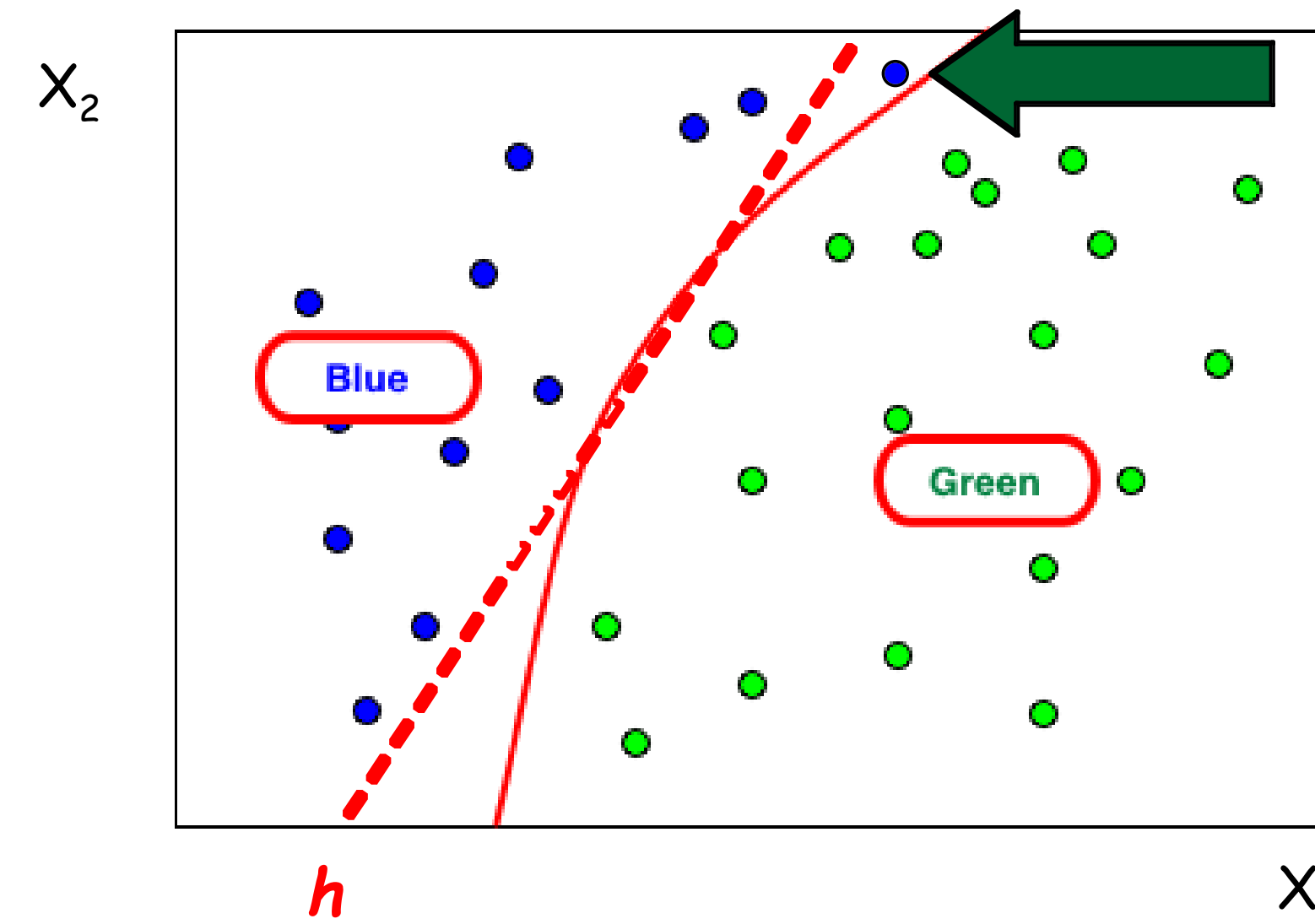
Focus of this course

CLASSIFICATION OUTPUT

- ▶ Different classification tasks can require different kinds of output
- ▶ **Class labels** – Each instance is assigned a single label
 - ▶ *Model only need to decide on crisp class boundaries*
- ▶ **Ranking** – Instances are ranked according to their likelihood of belonging to a particular class
 - ▶ *Model implicitly explores many potential class boundaries*
- ▶ **Probabilities** – Instances are assigned class probabilities $p(y|\mathbf{x})$
 - ▶ *Allows for more refined reasoning about sets of instances*
- ▶ Each requires progressively more accurate models (e.g., a poor probability estimator can still produce an accurate ranking)

DISCRIMINATIVE CLASSIFICATION

- ▶ Output: Class Labels
 - ▶ Direct mapping from inputs $\mathbf{x}$ to class label y
 - ▶ No attempt to model probability distributions
- ▶ Model the decision boundary directly
- ▶ May seek a discriminant function $f(\mathbf{x}; \theta)$ that maximizes measure of separation between classes
- ▶ Examples:
 - ▶ Perceptrons, decision trees, nearest neighbor classifiers, support vector machines



PROBABILISTIC CLASSIFICATION

- ▶ Output: Probabilities
 - ▶ Maps from inputs $\mathbf{x}$ to class label y indirectly through posterior class distribution $p(y|\mathbf{x})$
- ▶ Model the underlying probability distributions
 - ▶ Posterior class probabilities: $p(y|\mathbf{x})$
 - ▶ Class-conditional and class prior: $p(\mathbf{x}|y)$ and $p(y)$
- ▶ Examples:
 - ▶ Naive Bayes classifier, logistic regression

KNOWLEDGE REPRESENTATION

KNOWLEDGE REPRESENTATION

- ▶ Underlying structure of the model or patterns that we seek from the data
- ▶ Model: high-level global description of dataset
 - ▶ Choice of model family determines **space** of parameters and structure
 - ▶ Estimate model parameters and possibly model structure from data

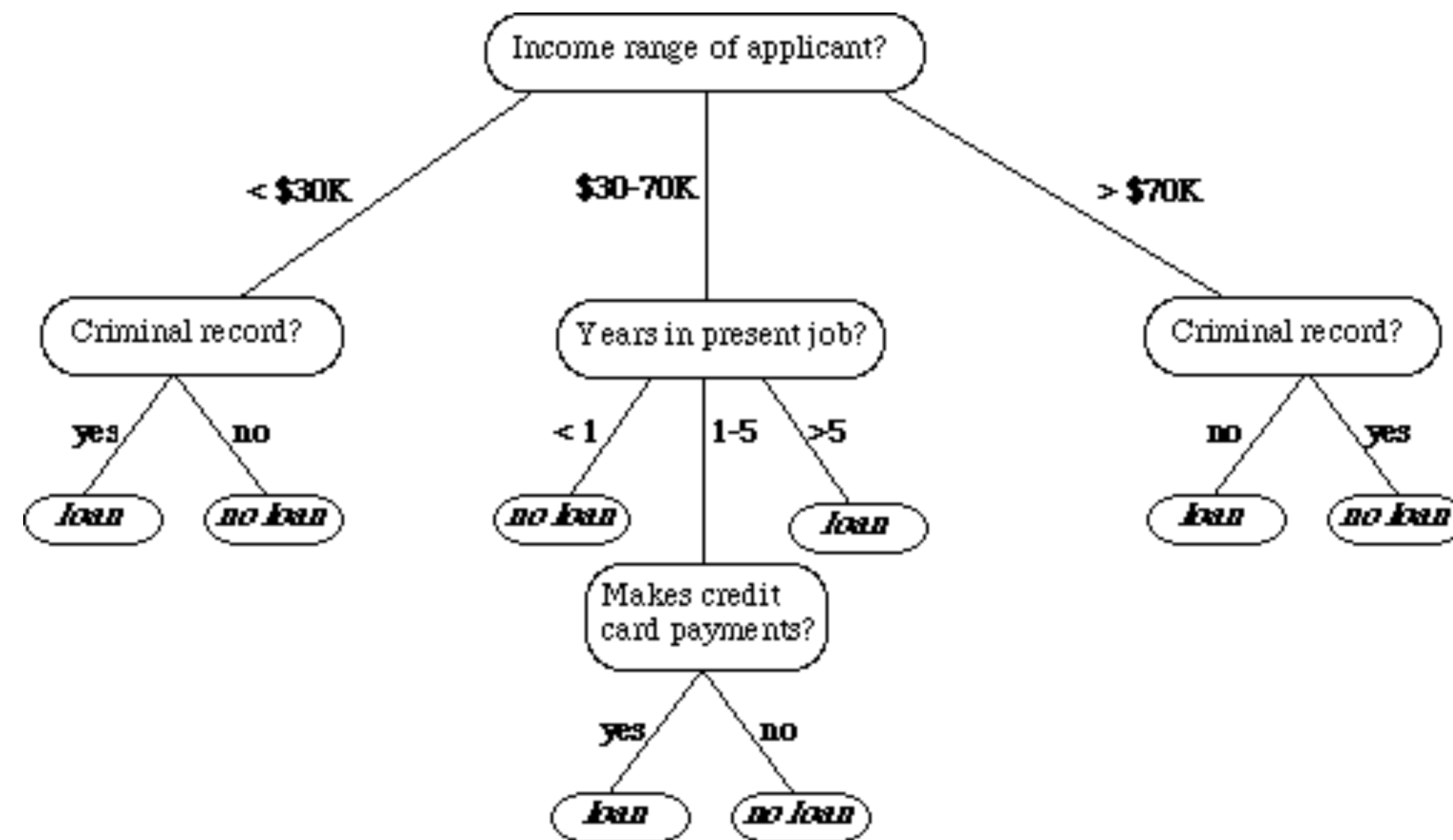
PERCEPTRON

$$f(x) = \begin{cases} 1 & \sum w_j x_j > 0 \\ 0 & \sum w_j x_j \leq 0 \end{cases}$$

Model space:

weights w , for each of j attributes

DECISION TREE



Model space:
all possible decision trees

MODEL SPACE

- ▶ How large is the space?
- ▶ Simplifying assumptions
 - ▶ Binary tree
 - ▶ Fixed depth
 - ▶ 10 binary attributes
- ▶ Can we search exhaustively?

Tree depth	Number of trees
1	10
2	8×10^2
3	3×10^6
4	2×10^{13}
5	5×10^{25}

NEAREST NEIGHBOR

Rule: find k closest (training) points to the test instance and assign the most frequently occurring class



Model space:

Choice of k , definition of distance, etc.

NAIVE BAYES CLASSIFIER

$$\begin{aligned} p(y|\mathbf{x}) &= \frac{p(\mathbf{x}|y)p(y)}{p(\mathbf{x})} \\ &= \frac{\prod_i p(x_i|y) p(y)}{\sum_j p(\mathbf{x}|y_j)p(y_j)} \end{aligned}$$

Model space:

parameters in conditional distributions $p(x_i|y)$

parameters in prior distribution $p(y)$